

MediVision

CNN-Based Multi-Domain Medical Image Screening

We built a system that looks at a photo of your skin, eye, or teeth and tells you what might be going on — automatically.

A

Arnav Mehta

14552020

A

Arpita Pandit

24603351

Y

Yash Hingu

14552032

Why we built MediVision

The gap between having a symptom and seeing a specialist

The problem

You notice a weird spot on your skin, or your eye looks red, or a tooth hurts. What do you do?

Most people wait. Dermatologists are hard to get, eye clinic waitlists are long, and dental visits are expensive. By the time you actually see someone, things may have gotten worse.

Existing AI tools each handle only one area — a skin app, or a retina scanner. None of them work across domains.

24

conditions detected

3

clinical domains

Our approach

One upload, three domains. The user doesn't pick "skin" or "eye" — the system figures it out automatically and routes to the right classifier.

We cover 24 conditions across skin (11), eye (6), and teeth (7), with Grad-CAM explanations so the user can actually see why the model made its call.

Think of it like a GP triage — not replacing doctors, just helping people decide if they should go see one.

35,884

training images

85.9%

end-to-end accuracy

Three questions we had to answer

01

Can CNNs actually work here?

Skin lesions, eye conditions, dental issues — can deep learning reach useful accuracy on public data alone?

Yes — transfer learning from ImageNet gets us there. ResNet50 hits 84-96% per domain.

02

Can one model route all three?

Skin photos look nothing like dental X-rays. Can a single classifier reliably tell them apart?

A dedicated domain router with argmax gets 99.45% routing accuracy on 5,648 test images.

03

How do we keep users safe?

If the model is wrong and someone acts on it, that's a real problem. How do we handle uncertainty?

Grad-CAM shows what the model sees, calibrated abstention flags low-confidence cases, and a disclaimer gate blocks everything until acknowledged.

The data we worked with

Pulled from 7 public sources on Kaggle & Roboflow, then cleaned pretty aggressively.

Skin

11 classes

Melanoma, BCC, Eczema, Psoriasis, Nevus, Tinea...

~18.5K images

Eye

6 classes

Cataract, Conjunctivitis, Glaucoma, Uveitis, Dry Eye...

~1.9K images

Teeth

7 classes

Caries, Gingivitis, Calculus, Hypodontia, Ulcer...

~11K images

What we did to clean it up

PIL magic-byte check

Forced full pixel decode on every file — caught mislabelled formats and truncated images

Perceptual hash dedup

16×16 grayscale hashing removed 5,855 duplicates (28% of the teeth set was duplicated!)

Stratified splitting

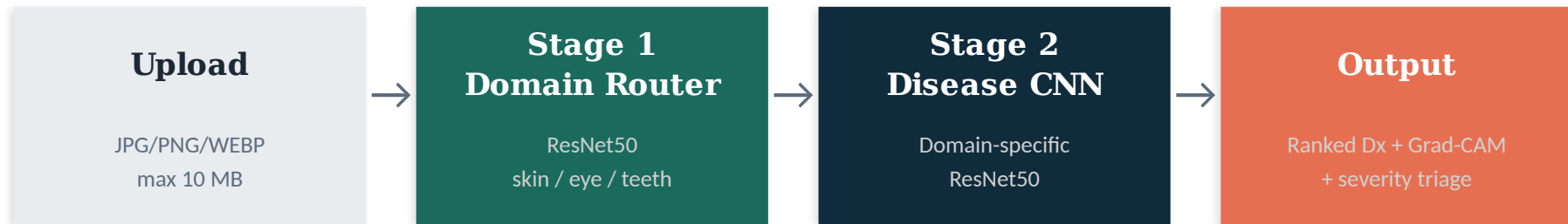
70/15/15 with leakage checks — no image appears in more than one split

Class rebalancing

Capped dominant Nevus class, upsampled minorities, focal loss for skin, NST augmentation for rare eye classes

How the pipeline works

Two stages — first figure out what kind of image it is, then diagnose it.



Why not just one 24-class model?

Each domain gets its own settings

Eye uses 384×384 inputs, skin uses focal loss, teeth uses standard CE. A flat model can't do that.

No cross-domain interference

Tuning the skin loss function doesn't accidentally hurt teeth performance.

Easy to swap parts out

We can retrain just the eye model without touching the others. Much more practical.

Why we picked ResNet50 over VGG16

We trained both architectures on all four models (8 models total) and compared.

Metric	VGG16	ResNet50
End-to-end accuracy	84.03%	85.91%
Skin (11-class, hardest)	82.18%	83.94%
Eye (6-class)	93.75%	93.99%
Teeth (7-class)	95.71%	95.71%
Domain routing	98.96%	99.56%
Skin Macro F1	0.753	0.775
Feature depth	7×7×512	7×7×2048

The bottom line

ResNet50 wins on every metric.

The biggest gap is on skin — the hardest task — where the higher macro F1 means better recall on rare malignant classes like melanoma and BCC.

Skip connections give smoother gradients during fine-tuning, and the 4× richer feature maps help distinguish visually similar conditions.

It's also the industry standard for medical imaging, so easier to benchmark.

How we trained each model

Not one-size-fits-all — each domain needed different treatment.

Domain Router

224×224 | Single-phase | CE + label smoothing

Full backbone unfrozen from epoch 0, LR 1e-4, patience 10.
Straightforward — big dataset, clear visual differences between domains.

Skin Classifier

224×224 | Single-phase | Focal loss ($\gamma=2$)

Focal loss down-weights easy examples so the model focuses on the rare malignant classes. Minority classes upsampled to 2K; Nevus capped at 3.5K.

Eye Classifier

384×384 | Two-phase TL | CE + label smoothing

Smallest dataset (~1,940 samples). Phase 1: froze backbone, trained head 15 epochs. Phase 2: unfroze top 50 layers at LR 1e-5. NST augmentation for Dry Eye & Uveitis.

Teeth Classifier

224×224 | Single-phase | CE + label smoothing

Similar to domain router. Balanced class weights via sklearn. Generic augmentation (flips, rotation, zoom).

Common: Adam optimizer, Dropout 0.4, L2 1e-4, label smoothing 0.1, LR warm-up 3 epochs, balanced class weights, trained on A100 GPU

What the user actually sees

Flask backend + plain HTML/CSS/JS. No framework, no build step. Loads all 4 models at startup.

Landing Page

- First-visit disclaimer modal blocks everything
- Three domain cards show what we detect
- Amber medical-advice banner throughout
- Consent stored in sessionStorage

Upload Screen

- Drag-and-drop zone for JPG/PNG/WEBP (10 MB max)
- "Auto-detection enabled" banner — no manual domain pick
- Optional symptoms text box
- Client-side validation before anything hits the server

Results Screen

- Domain pill + confidence pill (turns red if below threshold)
- Top-k bar chart with all classes in the domain
- Grad-CAM heat-map overlay with activation legend
- Severity triage: Urgent (red) / Moderate (amber) / Mild (green)

Privacy: images processed in memory via BytesIO — never written to disk.

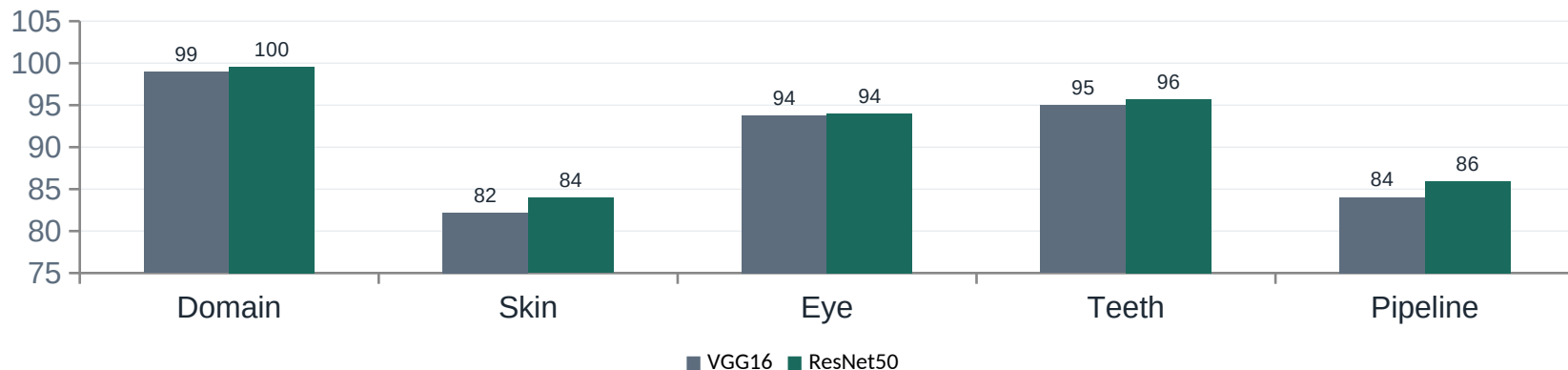
Results — what we actually achieved

85.91%

End-to-end pipeline accuracy
5,648-image stress test

Domain Routing	99.45%	Only 3 misroutes in 675 test images
Skin (11 classes)	83.94%	Melanoma F1: 0.97, BCC F1: 0.92
Eye (6 classes)	93.99%	Top-3 accuracy: 99.76%
Teeth (7 classes)	95.71%	Top-3 accuracy: 99.45%

All results with bootstrap 95% CIs (1,000 resamples)



Making it safe to use

A wrong prediction in a medical context can be dangerous. Here's how we handle that.

Grad-CAM heat-maps

Every prediction comes with a visual overlay showing exactly which pixels the model focused on. So if it's looking at the wrong part of the image, you can tell immediately. We used Grad-CAM on the last conv layer of each ResNet50.

Calibrated abstention

If the model's top probability falls below a threshold (0.80 for skin/teeth, 0.60 for eye), it flags the result as low-confidence instead of showing a confident-looking wrong answer. The confidence pill turns red and a warning banner appears.

Mandatory disclaimer gate

On first visit, a full-page modal blocks all interaction until the user acknowledges that this is not medical advice. Every page also carries a persistent amber disclaimer banner. The results screen explicitly recommends seeing a doctor.

Severity triage:

URGENT

MODERATE

MILD / NORMAL

What we'd do differently next time

Being honest about the gaps.

Known limitations

Caries recall is only 0.53

Precision is perfect (1.00) but we miss about half of actual caries cases. Only 36 test samples — we need more data.

Skin is tough at ~84%

Inflammatory conditions like eczema and psoriasis look similar even to dermatologists. Our biggest accuracy bottleneck.

No external validation

All metrics come from held-out splits of the same datasets. Real clinical performance could differ.

Routing could be fragile

The router may partly rely on lighting/camera cues rather than anatomy. Out-of-distribution phone photos might route wrong.

If we had more time

1. External clinical validation on independent datasets
2. Try newer backbones — EfficientNet-V2, ConvNeXt, maybe even ViT
3. Get more caries data with consistent intraoral photography
4. Turn on test-time augmentation in production (+0.5–1.5pp expected)
5. Explore a shared-backbone multi-task model to cut GPU memory by ~70%

Live Demo

Let's see MediVision in action

1

Upload a skin, eye, or dental
image

2

Watch Stage 1 auto-detect
the domain

3

See the ranked predictions
and confidence

4

Inspect the Grad-CAM
overlay and severity triage

Thank You

Questions & Oral Defense

CNN-Based Multi-Domain Medical Image Screening | Project 85 | 42028 Deep Learning & CNNs



Arnav Mehta

14552020



Arpita Pandit

24603351



Yash Hingu

14552032